ДИСЦИПЛИНА	ПРОГРАММИРОВАНИЕ КОРПОРАТИВНЫХ СИСТЕМ
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Ящун Татьяна Викторовна
СЕМЕСТР	6 семестр, 2025/26 уч. г.

ПРАКТИЧЕСКАЯ РАБОТА № 5
ЭФФЕКТИВНАЯ РАБОТА С ФУНКЦИЯМИ PUZZLE RPA

Цель работы: сформировать у студентов понимание функционального подхода в разработке RPA-процессов и научить применять функции в Puzzle RPA для оптимизации архитектуры робота, повторного использования логики и повышения читаемости алгоритма.

СОДЕРЖАНИЕ

ПОРЯДОК ОФОРМЛЕНИЯ РАБОТЫ 2
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ 2
Понятие функции в RPA 3
Основные характеристики функции в RPA 4
Структура функции в RPA 6
Использование функций в Puzzle RPA 8
ПРИМЕРЫ ЗАДАЧ. АВТОМАТИЗАЦИЯ ОТБОРА СОТРУДНИКОВ В КОРПОРАТИВНУЮ АКАДЕМИЮ 17
Общее условие 17
Требования к исходным данным: 17
Задача 1 18
Задача 2. Формирование списка рекомендованных к обучению 21
Результаты работы 24
КОНТРОЛЬНЫЕ ЗАДАНИЯ 25
Общая часть задания практической работы 25
Требования к выполнению индивидуальных заданий 26
Варианты заданий 28
ПОРЯДОК ЗАЩИТЫ ПРАКТИЧЕСКОЙ РАБОТЫ 29

ПОРЯДОК ОФОРМЛЕНИЯ РАБОТЫ

1. Откройте текстовый редактор, например, MS Word, и создайте текстовый документ.
2. Сохраните документ под именем *PR5_Surname_RPA_Function* где *Surname* – Ваша фамилия.
3. Добавьте в документ титульный лист отчета по Практической работе.
4. Внесите изменения в титульный лист, указав: группу, ФИО, вариант.
5. Добавьте в отчет следующие разделы:

- цель практической работы;
- теоретический материал (сделайте полную копию раздела «Теоретическая часть» из методички);
- **условие задания Вашего варианта (скопируйте условие из учебного репозитория);**
- краткое описание алгоритма решения каждого из 2-х заданий Вашего варианта;
- скриншот схемы алгоритмов работы робота для каждого из 2-х заданий Вашего варианта (**на скриншоте зафиксировать кроме схемы работы робота имя проекта, содержащее Вашу фамилию**);
- скриншот результата выполнения алгоритма для каждого из 2-х заданий Вашего варианта;
- **публичную ссылку на чат Вашей защиты практической работы с помощью ИИ DeepSeek.**

6. После выполнения и оформления всех заданий сохраните отчет под указанным выше именем в формате *.pdf*.
7. По результатам выполнения работы представьте отчёт по практической работе, содержащий, кроме содержательной части, ссылку на репозиторий с файлами проектов Puzzle RPA.
8. Прикрепите сформированный отчет на СДО.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Современные системы RPA позволяют создавать программных роботов для выполнения повторяющихся бизнес-операций. Однако по мере усложнения задач возникает необходимость структурирования алгоритмов, разделения логики и повышения управляемости процесса.

Одним из ключевых инструментов структурирования является использование функций.

Изучение функций в Puzzle RPA – это момент, когда вы переходите от простого создания роботов к настоящему проектированию автоматизированных решений.

До этого этапа алгоритмы могли выглядеть как последовательность действий. После освоения функций вы начнёте мыслить как разработчик: структурировать процесс, продумывать архитектуру и создавать универсальные решения,

которые можно масштабировать и совершенствовать.

Функции открывают новые возможности:

- вы научитесь разбивать сложную задачу на понятные управляемые части;
- сможете создавать «умные» модули, которые работают независимо;
- перестанете дублировать действия и упростите логику процессов;
- сможете создавать решения, которые выглядят профессионально и уверенно защищаются.

Это шаг от учебного алгоритма к инженерному подходу.

После изучения темы вы сможете:

- строить архитектуру робота осознанно;
- создавать масштабируемые процессы;
- легко вносить изменения в проекты;
- автоматизировать более сложные бизнес-сценарии;
- чувствовать уверенность при защите своей работы.

Функции – это инструмент, который превращает исполнителя алгоритма в проектировщика системы.

Именно здесь начинается уровень профессиональной разработки в RPA.

Понятие функции в RPA

В RPA функция – это самостоятельный логический блок алгоритма, предназначенный для выполнения конкретной задачи в рамках общего процесса.

Функция позволяет выделить часть действий в отдельный модуль, который можно:

- вызывать при необходимости;
- использовать повторно;
- изменять независимо от основного процесса;
- тестировать отдельно.

Проще говоря, функция – это «отдельный механизм» внутри робота, выполняющий одну чётко определённую операцию.

В RPA-решениях процессы часто включают:

- чтение данных;
- проверку условий;
- обработку информации;
- формирование отчётов;
- запись результатов.

Если вся логика записана последовательно в одном блоке, алгоритм становится громоздким и сложным для понимания.

Функции позволяют:

- структурировать процесс;
- разделить ответственность между модулями;
- упростить сопровождение проекта;
- повысить устойчивость робота к изменениям.

Таким образом, функция является инструментом архитектурного построения автоматизированного решения.

Основные характеристики функции в RPA

Функция в роботизированной автоматизации процессов – это не просто удобный инструмент, а фундаментальный элемент архитектуры. Её характеристики определяют устойчивость, гибкость и профессиональный уровень разработки.

Рассмотрим ключевые свойства функции подробно.

1. *Самостоятельность* (Завершённость логики)

Каждая функция должна выполнять одну конкретную задачу и доводить её до логического завершения.

Принцип: «Одна функция – одна ответственность».

Например:

- функция проверки листа – только проверяет;
- функция обработки строки – только обрабатывает строку;
- функция записи – только записывает данные.

Принцип единственной ответственности (Single Responsibility Principle) является одним из базовых принципов качественной архитектуры. Его нарушение приводит к усложнению поддержки и росту количества ошибок.

Если функция делает слишком много действий, она:

- становится трудно читаемой;
- усложняется при отладке;
- тяжело модифицируется.

2. *Изолированность* (Независимость)

Функция должна работать с теми данными, которые ей переданы, а не с внешними случайными переменными.

Это называется контролируемая область видимости данных.

Почему это важно? Если функция использует глобальные переменные:

- возрастает риск непредсказуемых ошибок;
- усложняется тестирование;
- повышается зависимость между блоками.

Большинство сложных ошибок в автоматизации возникают именно из-за неконтролируемого изменения глобальных переменных.

Изолированность делает функцию:

- безопасной;
- предсказуемой;
- повторно используемой.

3. *Повторное использование* (Reusable logic)

Одна из самых сильных сторон функции – возможность многократного вызова. Например:

- обработка 10 листов таблицы;
- проверка 100 строк;

- формирование отчёта по нескольким категориям.

Вместо копирования логики используется одна функция, вызываемая в цикле. Это уменьшает объём алгоритма и снижает вероятность ошибок.

4. *Управляемость входных данных*

Функция принимает параметры – и именно это делает её универсальной. Например, вместо: 5 блоков обработки разных листов создаётся: 1 функция обработки листа, параметр «название листа»

Это позволяет:

- быстро масштабировать процесс;
- легко добавлять новые элементы;
- избегать дублирования логики.

Чем меньше «жёстко прописанных» значений внутри функции, тем выше её гибкость.

5. *Возврат результата*

Функция может возвращать:

- логическое значение (True / False);
- число;
- строку;
- список;
- словарь;
- таблицу.

Возврат результата делает процесс управляемым.

Главный алгоритм получает конкретный итог выполнения функции и принимает решение о дальнейших действиях.

Возврат значений позволяет строить многоуровневую логику – когда одна функция управляет вызовом другой.

6. *Тестируемость*

Функцию можно протестировать отдельно от всего процесса. Это означает:

- проще находить ошибки;
- быстрее проводить отладку;
- легче проверять корректность расчётов.

В профессиональной разработке тестируемость является критерием качества архитектуры.

7. *Масштабируемость*

Если процесс увеличивается (больше листов, больше условий, больше данных), функции позволяют:

- изменить один блок без переписывания всего алгоритма;
- добавить новую функцию без разрушения структуры;
- адаптировать процесс под новые требования.

Роботы без функций плохо масштабируются. Роботы с функциями легко расширяются.

8. *Читаемость и архитектурная прозрачность*

Алгоритм, разделённый на функции, читается как последовательность шагов высокого уровня:

- получить данные;
- обработать данные;
- сформировать результат;
- записать результат.

Это делает процесс понятным не только разработчику, но и проверяющему или заказчику. Читаемость алгоритма напрямую влияет на скорость сопровождения проекта.

Структура функции в RPA

Функция – это не просто набор действий, а логически выстроенная конструкция с чёткой внутренней организацией.

Понимание её структуры позволяет проектировать устойчивые, масштабируемые и профессиональные автоматизированные процессы.

Любая функция в RPA имеет четыре ключевых элемента:

- Название
- Входные параметры
- Внутреннюю логику
- Возвращаемое значение

Рассмотрим каждый элемент подробно.

1. *Название функции*

Название – это первый уровень архитектуры. Оно должно:

- отражать назначение;
- быть конкретным;
- быть понятным без изучения внутренней логики;
- описывать действие, а не объект.

Пример хороших названий:

- `ValidateSheetName`
- `ProcessPaymentRow`
- `CalculateTotalAmount`
- `CheckDuplicateRecords`

Пример плохих названий:

- `Function1`
- `Test`
- `Data`
- `NewBlock`
- `MyProcess`

В инженерной разработке до 40% времени сопровождения уходит на понимание логики кода. Грамотные названия сокращают это время в несколько раз.

Название – это архитектурная подсказка для разработчика и проверяющего.

2. *Входные параметры*

Входные параметры – это данные, которые функция получает извне. Они делают функцию:

- универсальной;
- управляемой;
- независимой от жёстко заданных значений.

Например: Функция обработки листа не должна «знать» конкретное название листа внутри себя. Название должно передаваться параметром. Это позволяет использовать одну и ту же функцию для:

- разных листов;
- разных таблиц;
- разных наборов данных.

Почему это важно? Если значения «зашиты» внутри функции:

- её сложно изменить;
- её нельзя переиспользовать;
- возрастает риск ошибок при расширении проекта.

В профессиональной автоматизации жёстко заданные значения считаются одной из главных причин неустойчивости процессов.

3. *Внутренняя логика*

Это «сердце» функции – последовательность действий, которые она выполняет. Правильная внутренняя логика должна быть:

- последовательной;
- читаемой;
- ограниченной одной задачей;
- не перегруженной дополнительными обязанностями.

Важный принцип: Функция должна делать одно действие качественно, а не несколько действий частично.

Плохая практика:

Функция одновременно:

- читает таблицу,
- проверяет ошибки,
- формирует отчёт,
- записывает данные.

Хорошая практика: разделить это на несколько функций.

Чем длиннее функция, тем выше вероятность логической ошибки.

Функции с большим количеством логических ветвлений имеют значительно больший процент дефектов.

4. *Возвращаемое значение*

Возврат результата – это способ взаимодействия функции с главным процессом. Возврат результата позволяет:

- строить многоуровневую логику;
- принимать решения в главном алгоритме;

- контролировать успешность выполнения.

Пример логики: `Функция View_Sheet → возвращает True или False`

Главный процесс:

- если `True` – вызвать функцию обработки;
- если `False` – перейти к следующему листу.

Это создаёт управляемую архитектуру, а не хаотичную последовательность действий.

5. *Связь функции с главным процессом*

Функция не существует отдельно. Она вызывается главным алгоритмом. Правильная структура выглядит так:

- Главный процесс:
  - ✓ получает список листов;
  - ✓ вызывает функцию обработки;
  - ✓ получает результат;
  - ✓ принимает решение.
- Функция:
  - ✓ выполняет строго свою задачу;
  - ✓ возвращает результат;
  - ✓ не управляет всем процессом целиком.
- Главный процесс – координатор.
- Функции – исполнители.

Структура функции – это не просто формальность, а система, которая обеспечивает:

- модульность;
- масштабируемость;
- повторное использование;
- управляемость;
- читаемость;
- снижение количества ошибок.

Понимание структуры функции – это шаг от простого составления алгоритмов к инженерному проектированию автоматизированных решений.

Использование функций в Puzzle RPA

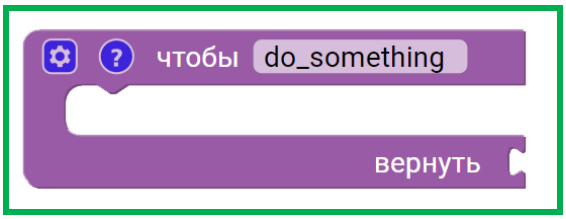
В Puzzle RPA функция – это самостоятельный логический блок, который выполняет конкретную задачу в рамках общего процесса.

Она создаётся внутри процесса как отдельный блок, может принимать входные параметры, содержать собственную логику и возвращать результат.

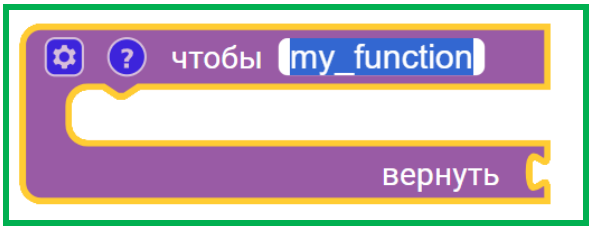
Все функции, используемые в роботах Puzzle RPA, находятся в разделе «Стандартные».

Чтобы использовать функции в Puzzle RPA требуется:

1. Создать функцию;

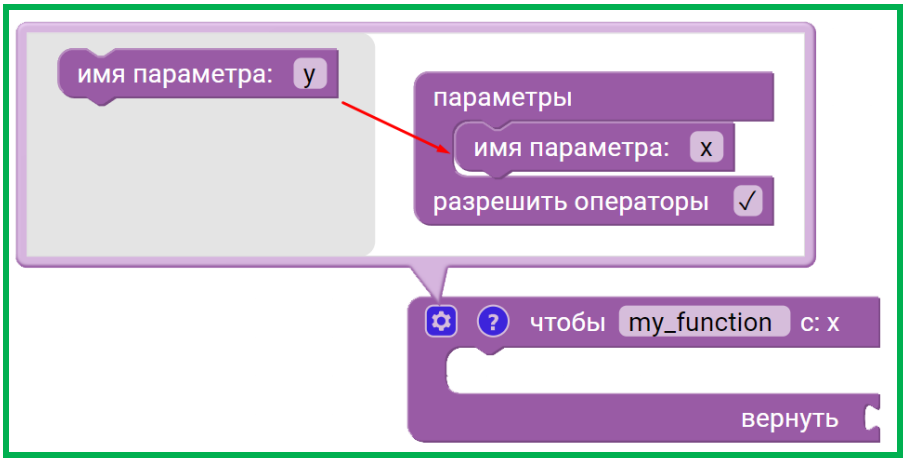


2. Указать название функции в поле ввода; В названии функции не допускаются пробелы.

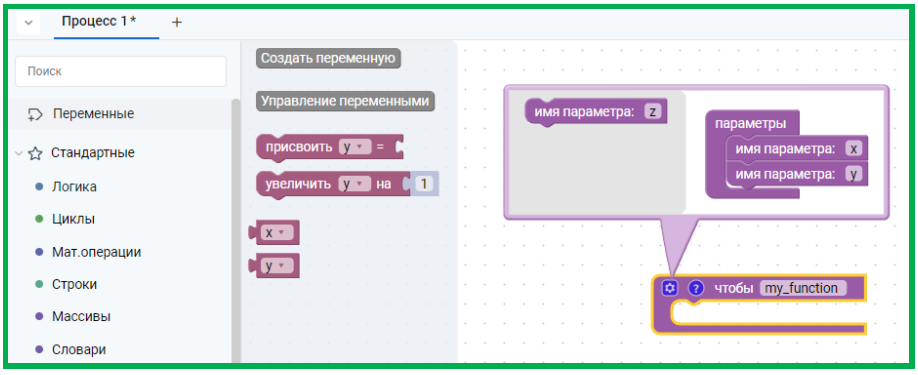


3. Добавить параметры, которые будут использоваться в алгоритме функции;

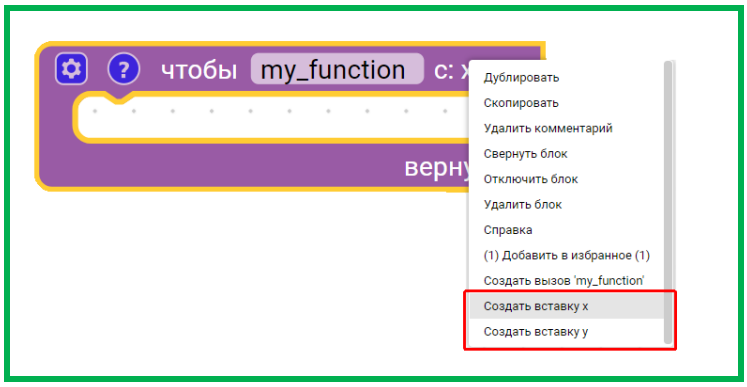
Параметры – это переменные, которые вы определяете в объявлении функции. Они служат для значений, которые будут переданы функции при её вызове. Параметры позволяют функции принимать входные данные и использовать их в своей логике.



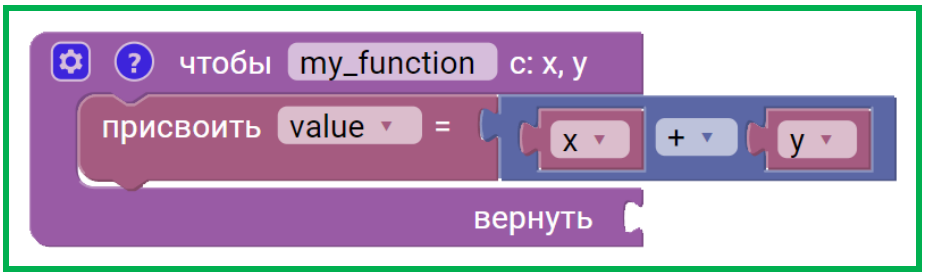
Переменные, используемые в качестве параметров функций, отображаются в списке переменных для всех процессов, независимо от того, в каком именно процессе объявлена функция.



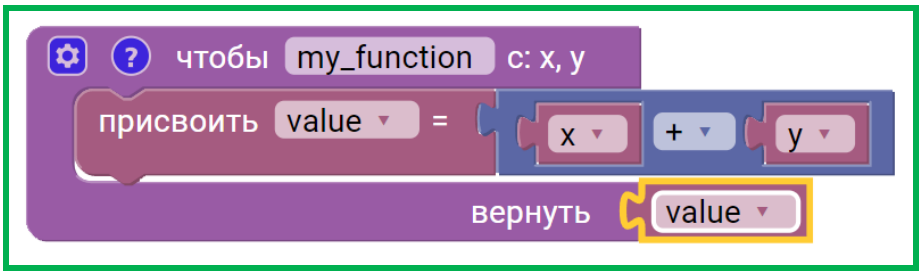
Для создания вставки переменной-параметра, необходимо открыть контекстное меню функции и кликнуть на кнопку **Создать вставку** соответствующего параметра:



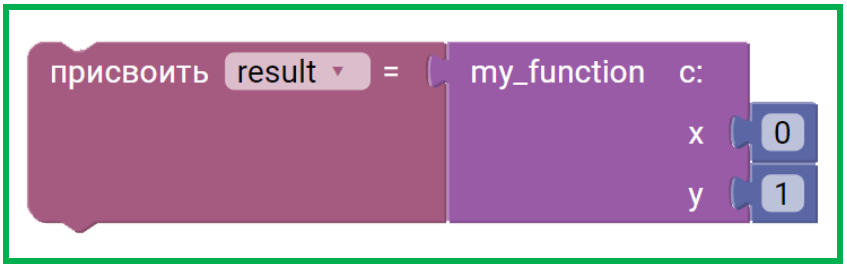
4. Собрать алгоритм внутри функции;



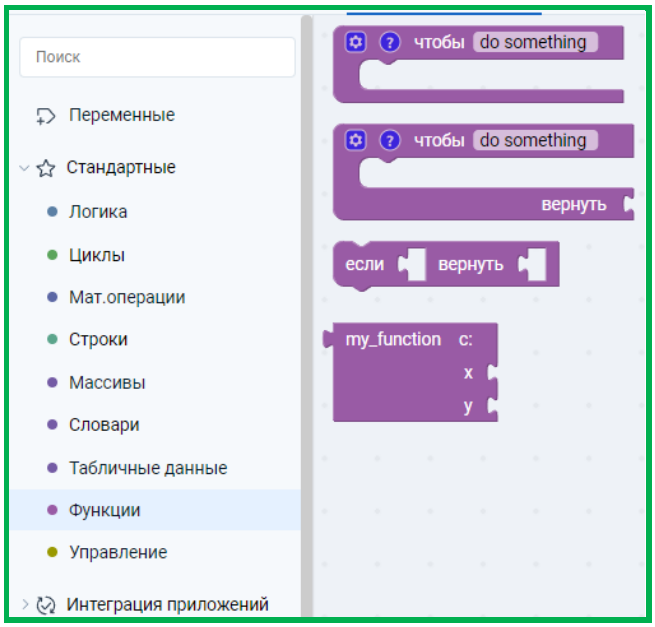
5. Указать значения, которые будут возвращены из функции, если это требуется по процессу;



6. Выполнить функцию в любом месте основного алгоритма.

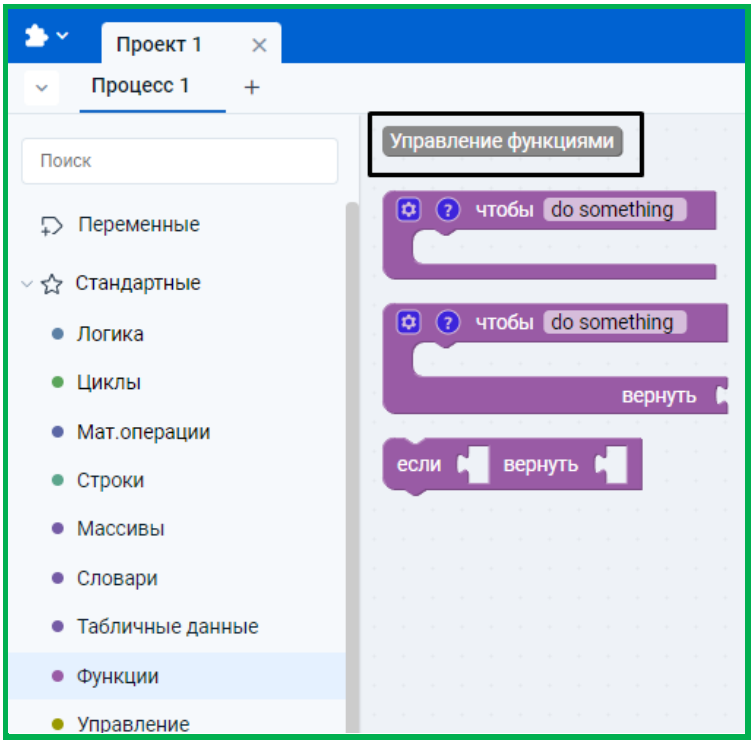


Для выполнения функции в основном алгоритме необходимо добавить блок вызова. Блок автоматически генерируется при создании функции в подкатегории “Функции”.



Функции, разработанные в дочерних процессах проекта, могут быть использованы в других процессах этого же проекта робота. При создании функции её вызов доступен во всех процессах, относящихся к одному проекту робота. Функции, объявленные в главном процессе, нельзя вызывать и использовать из дочерних процессов.

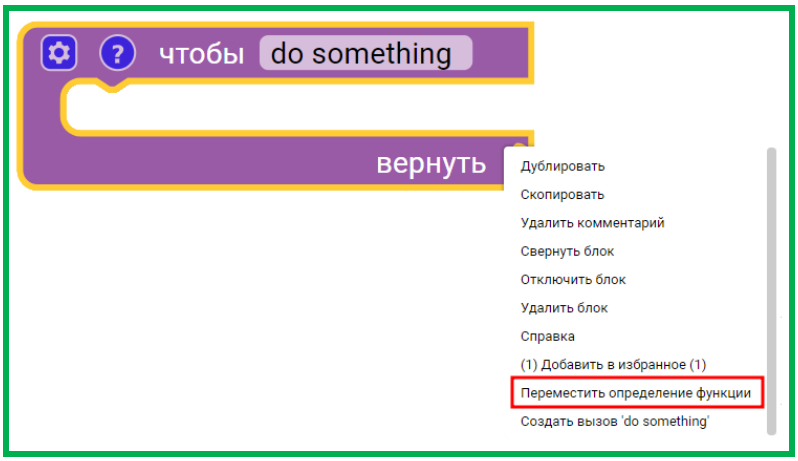
Клик на кнопку «Управление функциями» открывает встроенный инструмент студии Puzzle RPA, который отображает все функции и переменные, используемые в проекте.



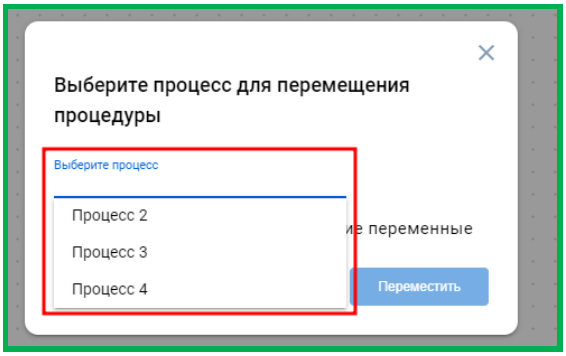
Список переменных и функций – встроенный инструмент Puzzle RPA, который отображает все функции и переменные, используемые в проекте.

Функции, разработанные в проекте, могут быть перенесены в другой процесс в данном проекте. Для этого требуется:

1. Кликнуть правой кнопкой мыши по блоку определения функции и выбрать пункт «Переместить определение функции».



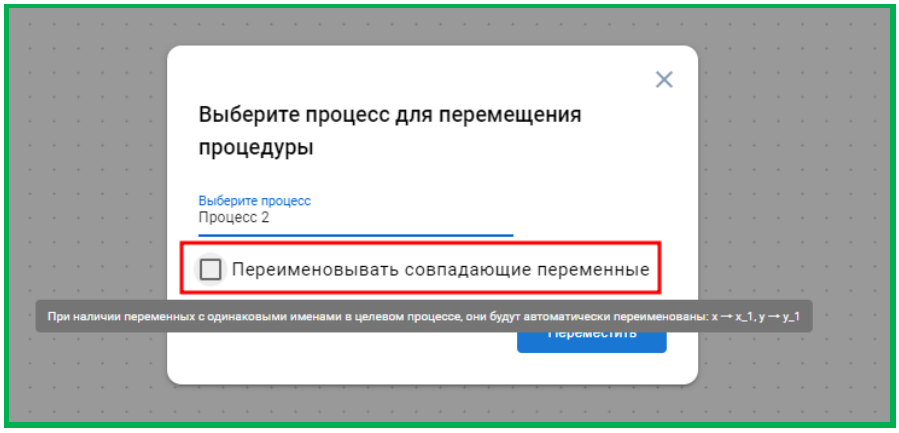
2. В открывшемся окне выбрать процесс, в который будет перенесена функция.



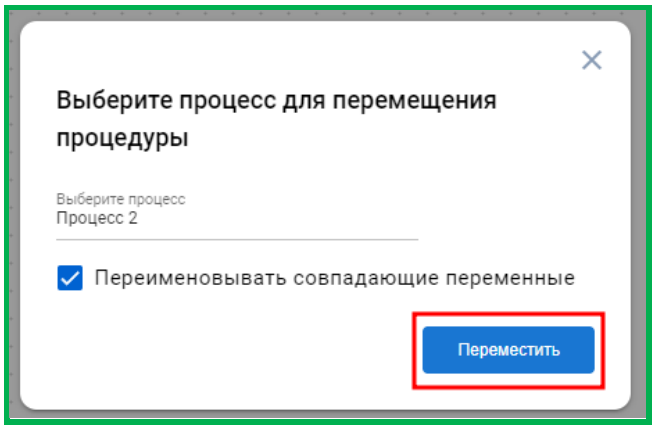
3. Определить поведение при совпадении имён переменных функции и переменных целевого процесса;

Если в целевом процессе уже существует переменная, имя которой используется в алгоритме функции, и выбрана опция «Переименовывать совпадающие переменные», то переменные в теле функции будут автоматически переименованы (например: x → x_1, y → y_1).

В случае совпадения имени параметра функции с переменной в целевом процессе, автоматическое переименование не применяется.



4. Выполнить клик на кнопку «Переместить» для подтверждения.

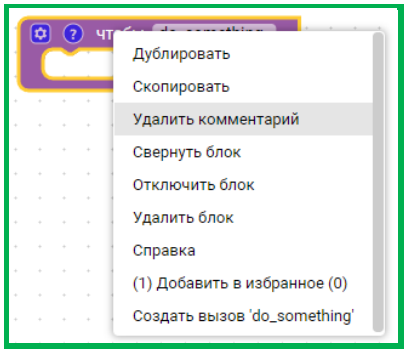


Удалить созданную функцию можно несколькими способами:

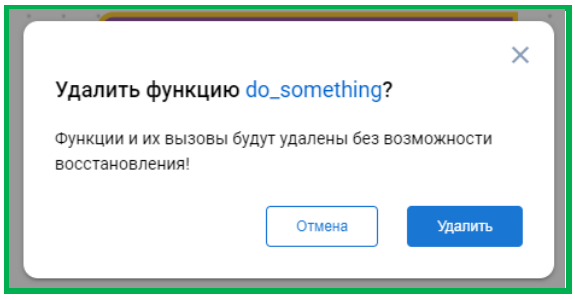
- Способ 1 – Через контекстное меню

  Для удаления через контекстное меню требуется:

  1. Выбрать функцию для удаления;
  2. Выполнить клик правой кнопкой мыши по блоку Определение функции;



  3. Кликнуть на вариант Удалить блок;
  4. Подтвердить удаление кликом на кнопку Удалить.



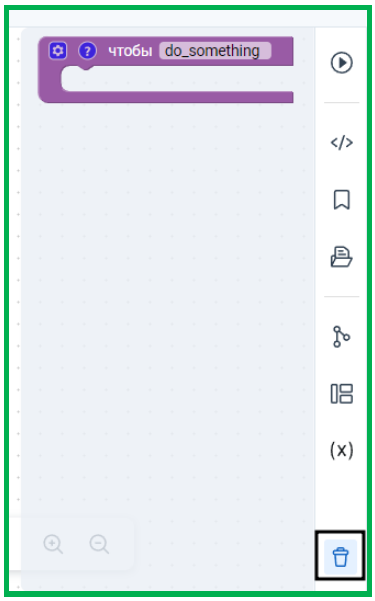
- Способ 2 – Через форму управления функциями

Клик на кнопку «Управление функциями» открывает встроенный инструмент студии Puzzle RPA, который отображает все функции и переменные, используемые в проекте. Этот инструмент позволяет выполнить удаление функций.

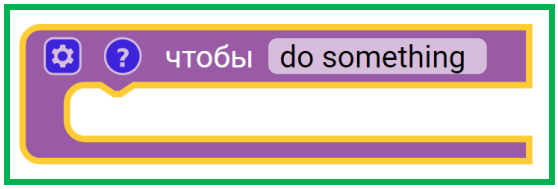
Невозможно:

- Отменить удаление функций комбинаций клавиш **Ctrl + Z**;
- Вырезать объявление функции комбинаций клавиш **Ctrl + X**.

Восстановить удаленную функцию можно только из корзины:



1. Блок Определение функции

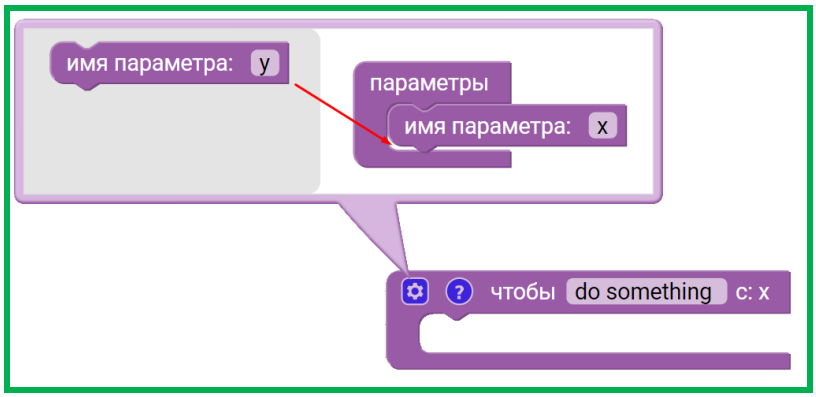


Блок создает процедуру, которая выполняет определенные действия, но не возвращает никакого значения.

Чтобы открыть форму управления параметрами функции, требуется кликнуть на кнопку:



Для добавления нового параметра в функцию необходимо перетащить его в область параметров в интерфейсе управления параметрами функции. При необходимости можно также изменить имя добавленного параметра.



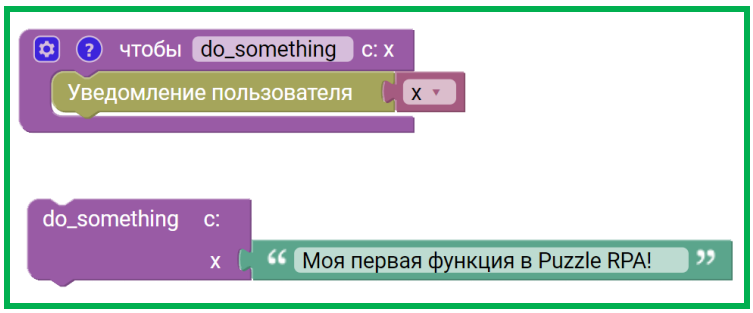
Для выполнения функции в основном алгоритме необходимо добавить блок вызова.



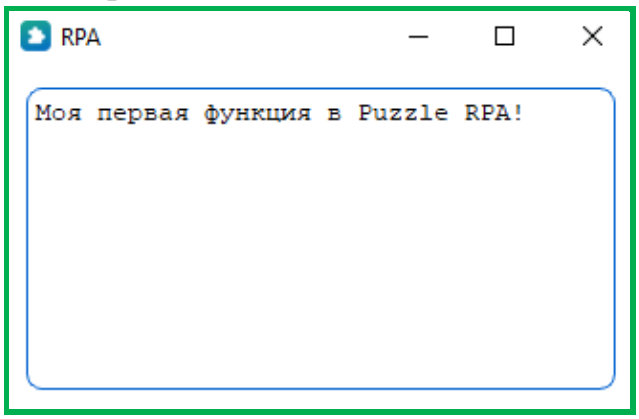
Блок автоматически генерируется при создании функции в подкатегории «Функции».

Пример использования

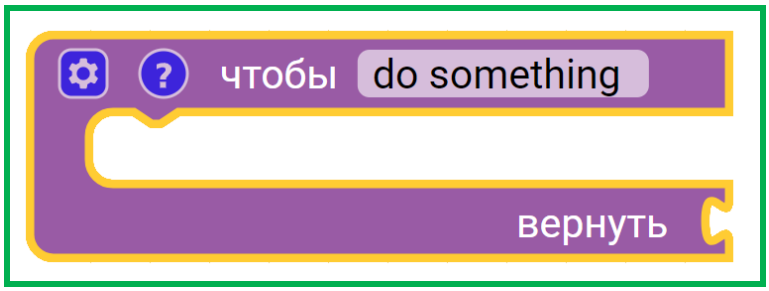
В данном примере создается функция `do_something`, которая принимает параметр `x`. Значение параметра выводится на экран через блок “Уведомление пользователя”. В качестве аргумента функции передается строка “Моя первая функция в Puzzle RPA!”.



Вывод результата на экран:



2. Блок Определение функции и возврат значения

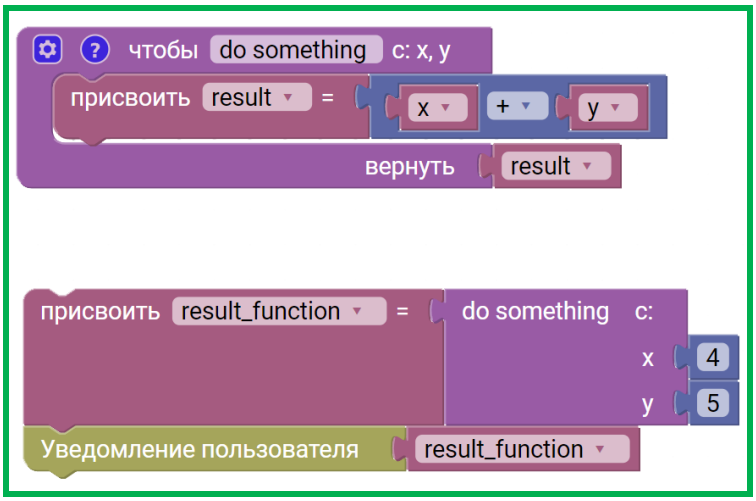


Блок создает процедуру, которая выполняет определенные действия и возвращает результат выполнения.

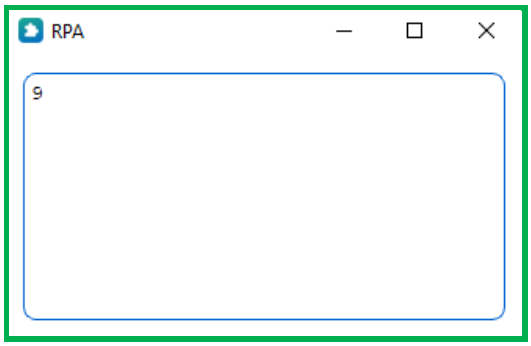
Пример использования:

В данном примере создается функция `do_something`, которая принимает параметр `x` и `y`. Внутри функции выполняется сложение двух принимаемых параметров. Результат возвращается из функции.

При вызове функции `do_something` в качестве аргументов передаются 2 числа: 4 и 5.



Вывод результата на экран:



3. Блок Если–вернуть

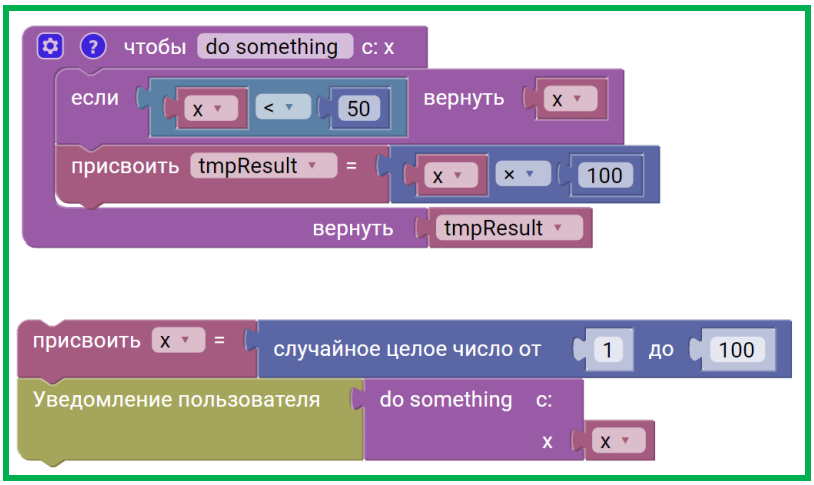


Блок помещается внутрь функции. Если условие внутри блока истина, то возвращается указанное значение.

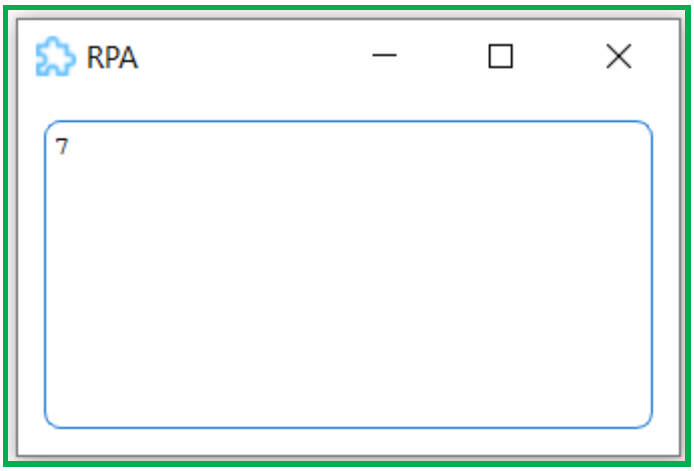
Пример использования

В данном примере блок возвращает число из функции без изменения, если случайное число меньше 50:

- Генерируется случайное число от 1 до 100;
- Вызывается функция, которая проверяет, если число менее 50, то возвращается число без изменения, в ином случае сгенерированное число увеличивается в 100 раз.



Возвращенное число без изменения:



ПРИМЕРЫ ЗАДАЧ. АВТОМАТИЗАЦИЯ ОТБОРА СОТРУДНИКОВ В КОРПОРАТИВНУЮ АКАДЕМИЮ

Общее условие

В крупной компании запускается внутренняя корпоративная академия. Руководство приняло решение ежегодно отбирать сотрудников для участия в программе обучения, направленной на развитие управленческих и профессиональных компетенций.

Ранее отбор проводился вручную: HR-специалисты анализировали возраст сотрудников, их рабочие показатели, количество выполненных задач и допущенных ошибок. Процесс занимал несколько дней и сопровождался риском ошибок при подсчётах.

В этом году принято решение автоматизировать процесс с использованием программного робота, разработанного в Puzzle RPA.

HR-отдел формирует список заявок сотрудников и передаёт его в виде JSON-файла. Задача разработчика – спроектировать робота с использованием функций, который:

- корректно считает данные из файла;
- структурированно обрабатывает список сотрудников;
- разделяет логику на самостоятельные модули;
- формирует новые списки результатов;
- сохраняет итоговые данные в отдельные JSON-файлы.

Требования к исходным данным:

- Исходные данные необходимо сгенерировать с помощью ИИ DeepSeek.
- Сформировать JSON-файл, содержащий ровно 6 записей.
- Формат – список словарей.
- Структура каждой записи:

```
{
  "name": "Иванов И.И.",
  "age": 25,
  "tasks_completed": 40,
  "errors": 3
}
```

Робот должен считать данные из созданного JSON-файла.

Задача 1

Формулировка задачи

На первом этапе руководство установило базовое требование: к обучению допускаются только совершеннолетние сотрудники.

HR-отделу необходимо получить отдельный файл со статусом допуска по каждому сотруднику.

Требуется:

- Считать JSON-файл (6 сотрудников).
- Организовать цикл обработки списка словарей.
- Создать функцию с возвратом значения.

  Функция `CheckEmployeeAge`

  Тип: Определение функции и возврат значения

  Параметр: `employee_dict`

  Логика:
- Получить значение age из словаря.

✓ Если возраст < 18 → вернуть "Недопущен".

✓ Если возраст ≥ 18 → вернуть "Допущен".

Главный процесс должен:

- Создать пустой список `admission_result_list`.
- В цикле:

✓ Вызвать `CheckEmployeeAge`;

✓ Сформировать словарь:

```
{
  "name": "Иванов И.И.",
  "admission_status": "Допущен"
}
```

– Добавить словарь в `admission_result_list`.

- Сохранить `admission_result_list` в файл `admission_result.json`.
- Вывести итоговый список на экран.

Архитектурные требования:

- Функция создаётся через блок «Определение функции и возврат значения».
- Функция выполняет только одну задачу – проверку возраста.
- Запрещено использовать глобальные переменные.
- Главный процесс не содержит логики проверки.

Этапы решения

Исходный JSON-файл с данными сотрудников мы заранее сгенерировали и поместили в папку resources проекта.

- Имя файла: `admission.json`
- Путь в проекте: `resources/ admission.json`
- Файл содержит 6 записей в формате списка словарей.

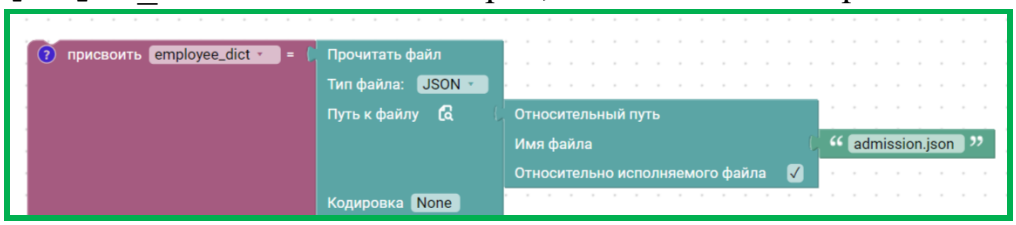
Шаг 1. Чтение исходного JSON

- Блок: «Прочитать файл»
- Файл: `admission.json`
- Результат: переменная `employee_dict`

Что мы получаем:

```
[
  {"name": "Иванов И.И.", "age": 25, "tasks_completed": 40, "errors": 3},
  {"name": "Петров П.П.", "age": 16, "tasks_completed": 30, "errors": 2},
  ...
]
```

`employee_dict` – список словарей, где каждый словарь – один сотрудник



Шаг 2. Создание функции `cheeck_employee_age`

- Блок: «Определение функции и возврат значения»
- Имя функции: `cheeck_employee_age`
- Параметр: `employee_list` (один словарь сотрудника)
- Логика функции

✓ Блоки внутри функции:

– Присвоить значение переменной

– `age = int(employee_list["age"])`

– Берём `age` из словаря сотрудника

– Условие (If)

```
Если age < 18:
    Вернуть "Недопущен"
Иначе:
    Вернуть "Допущен"
```

– Проверяем возраст

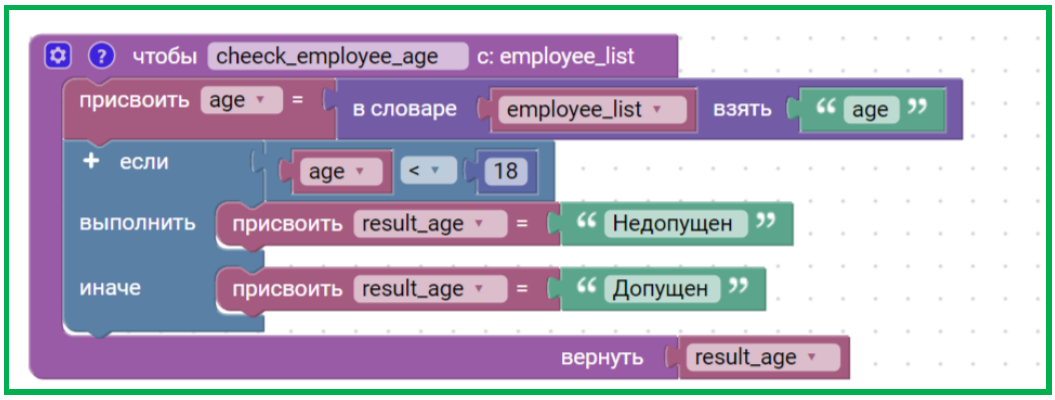
– Возвращаем строку со статусом

✓ Важно:

– Функция не содержит цикла

– Функция не работает со списком – только с одним словарём

– Только одна ответственность: проверка возраста



Шаг 3. Главный процесс

Цель: перебрать всех сотрудников и собрать результаты в список.

- 3.1 Создать пустой список

✓ Блок: «Присвоить значение»

✓ `admission_result_list = []`



- 3.2 Цикл по списку сотрудников

✓ Блок: «For Each employee_list in employees_dict»

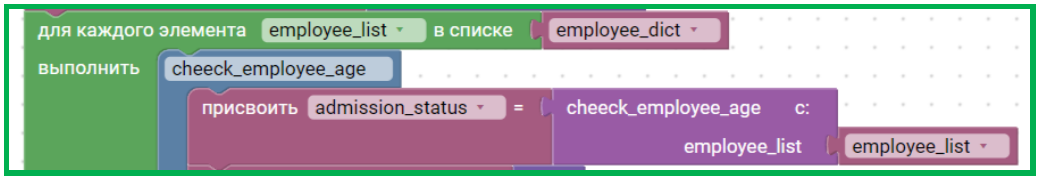
✓ Внутри цикла:

– Вызов функции

```
admission_status = CheckEmployeeAge(employee_list)
```

– Передаём один словарь сотрудника

– Получаем результат "Допущен" или "Недопущен"

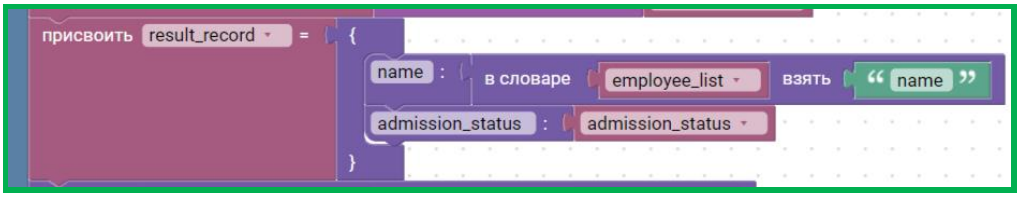


✓ Создание итогового словаря

```
result_record = {
    "name": employee_list["name"],
    "admission_status": admission_status
}
```

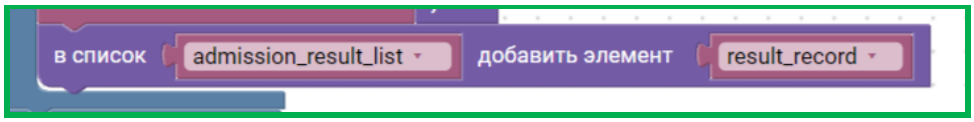
– Берём имя из словаря сотрудника

– Берём статус из функции



✓ Добавить словарь в список

✓ Добавить `result_record` в `admission_result_list`



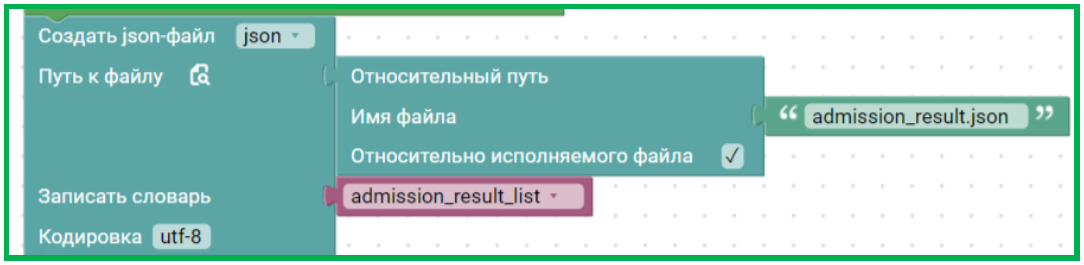
- 3.3 После цикла

✓ Сохранить список в JSON

✓ Блок: «Сохранить JSON-файл»

✓ Файл: admission_result.json

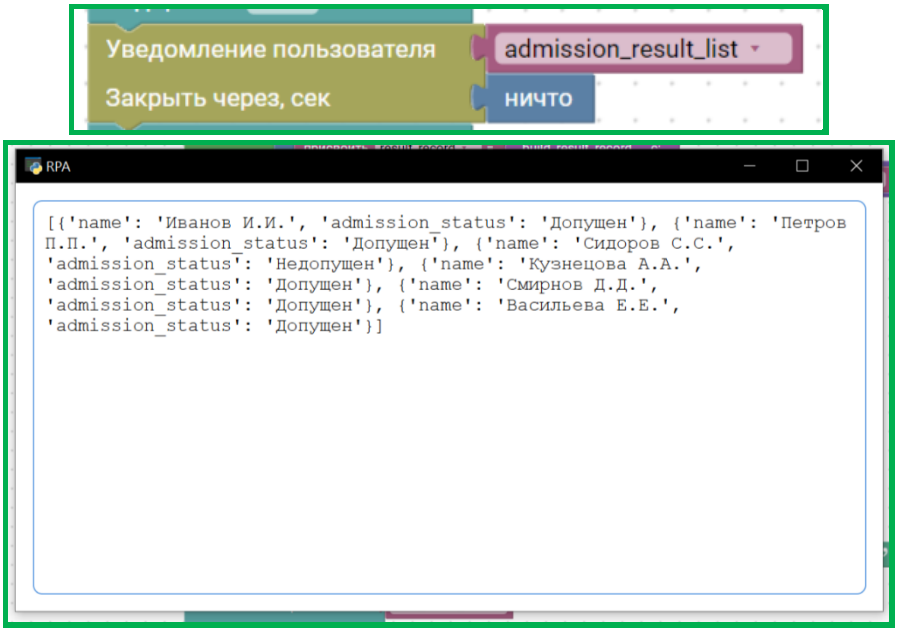
✓ Сохраняем admission_result_list



✓ Вывести итоговый список на экран

✓ Блок: «Вывести на экран»

✓ Переменная: `admission_result_list`



Задача 2. Формирование списка рекомендованных к обучению

Формулировка условия

Ситуация: После проверки возраста руководство компании решило усложнить процесс отбора.

Теперь требуется учитывать рабочую эффективность сотрудников. Было принято решение:

- оценивать соотношение выполненных задач к количеству ошибок;
- формировать рекомендацию на основании коэффициента эффективности;
- создать отдельный итоговый файл с результатами второго этапа отбора.

HR-отделу нужен структурированный список сотрудников со статусом рекомендации для передачи руководителям подразделений.

Требуется:

- Использовать исходный JSON-файл из 6 сотрудников.
- Организовать цикл обработки списка.

- Создать три функции с возвратом значения.

Важно: Используется тот же JSON-файл из предыдущего этапа, который мы заранее сгенерировали и поместили в папку resource проекта.

Этапы решения

- Шаг 1. Чтение исходного JSON

✓ Блок: «Прочитать файл»

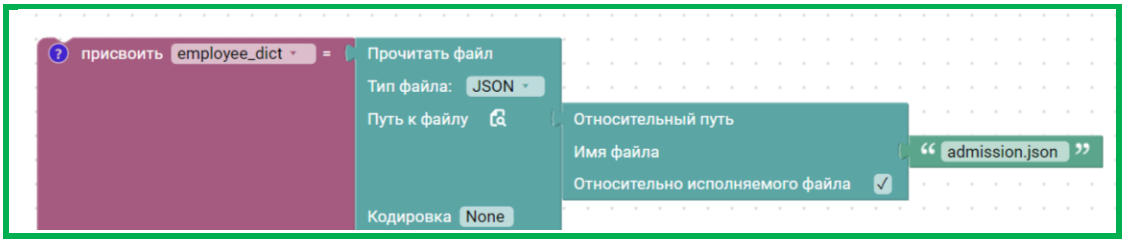
✓ Файл: admission.json

✓ Результат: переменная employee_dict

✓ Что мы получаем:

```
[
  {"name": "Иванов И.И.", "age": 25, "tasks_completed": 40, "errors": 3},
  {"name": "Петров П.П.", "age": 16, "tasks_completed": 30, "errors": 2},
  ...
]
```

✓ `employee_dict` – список словарей, где каждый словарь – один сотрудник



- Шаг 2. Функция calculate_performance

✓ Параметр: employee_list

✓ Логика:

– Получить `tasks_completed` и `errors`.

– Если `tasks_completed = 0` → вернуть 0.

– Рассчитать коэффициент: `(tasks_completed - errors) / tasks_completed`

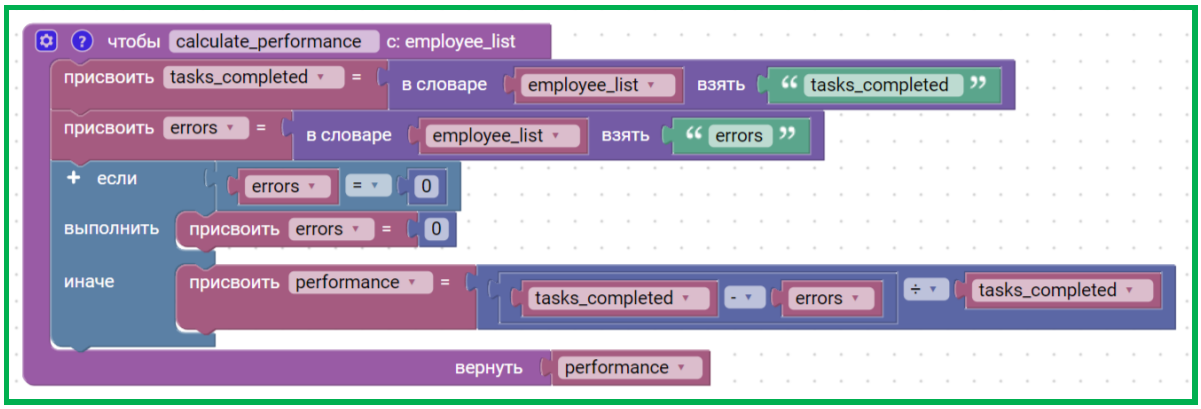
– Вернуть число.

Важно: Проверка на деление на ноль

✓ Блок: Условие

✓ Если `tasks_completed = 0`

Почему это важно? Если задач 0, делить нельзя – будет ошибка.



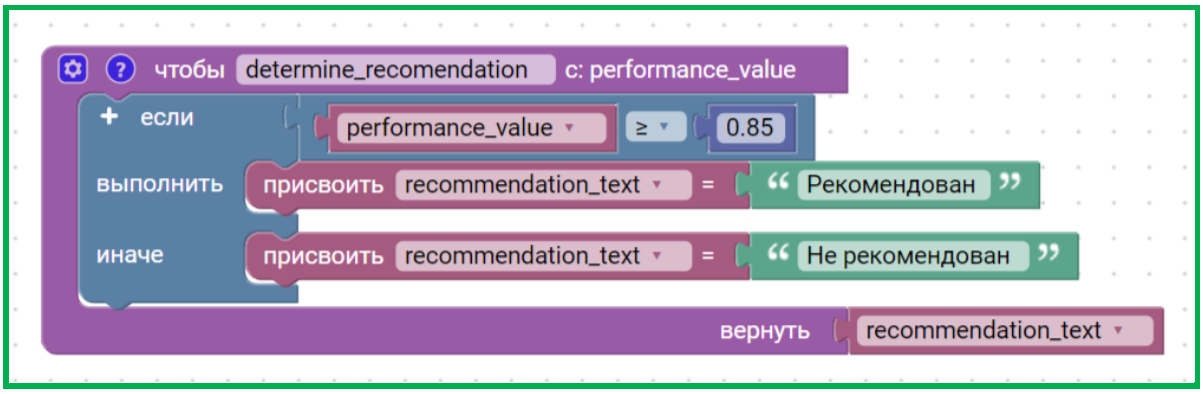
- Шаг 3. Функция determine_recomendation

✓ Параметр: performance_value

✓ Логика:

– Если коэффициент ≥ 0.85 → вернуть "Рекомендован"

– Иначе → вернуть "Не рекомендован"



- Шаг 4. Функция build_result_record

✓ Параметры:

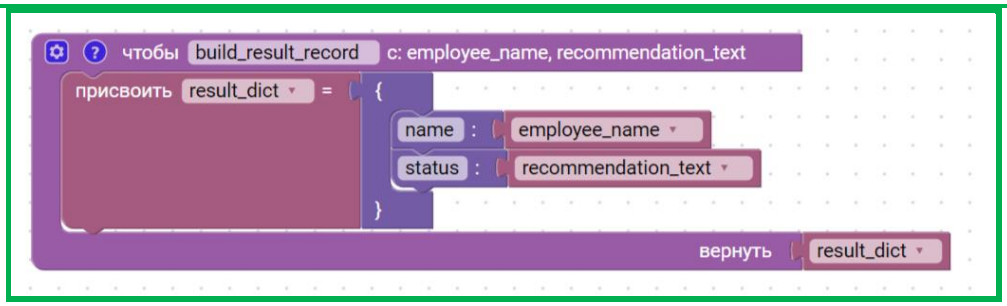
– employee_name

– recommendation_text

✓ Логика:

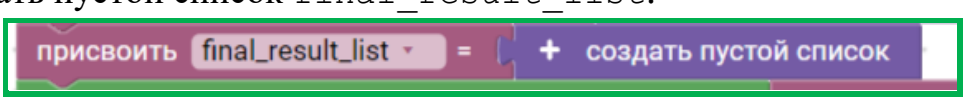
– Создать и вернуть словарь:

```
{
  "name": "Иванов И.И.",
  "status": "Рекомендован"
}
```



✓ Главный процесс должен:

– Создать пустой список `final_result_list`.

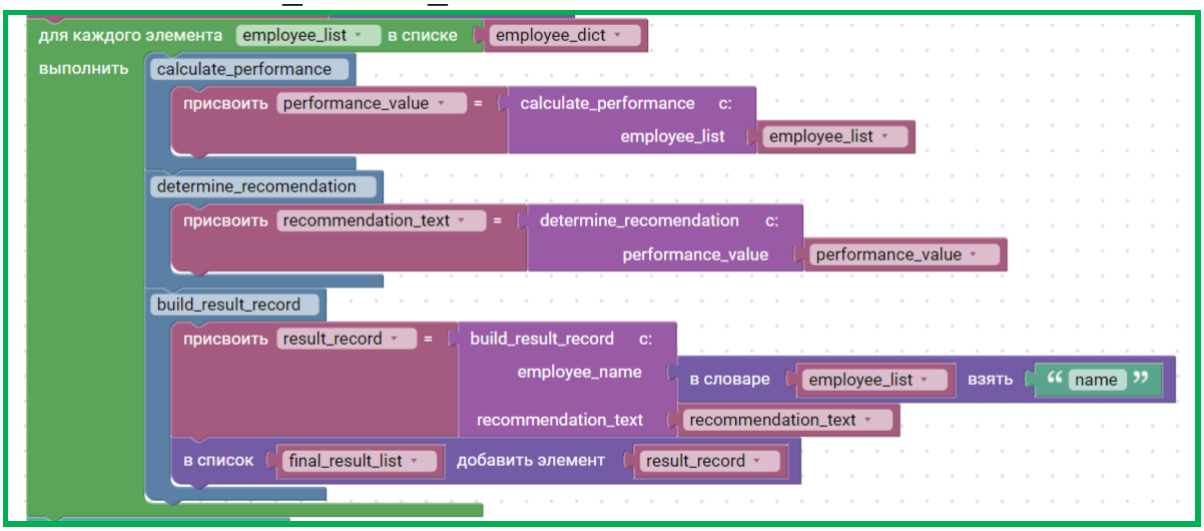


✓ В цикле для каждого сотрудника:

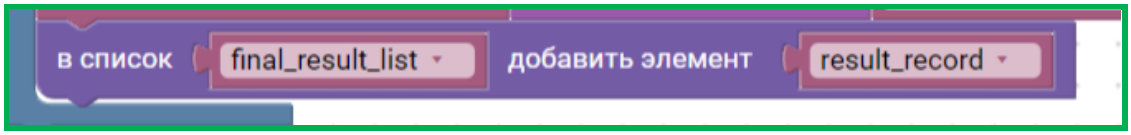
– Вызвать `calculate_performance`;

– Передать результат в `determine_recomendation`;

– Вызвать `build_result_record`

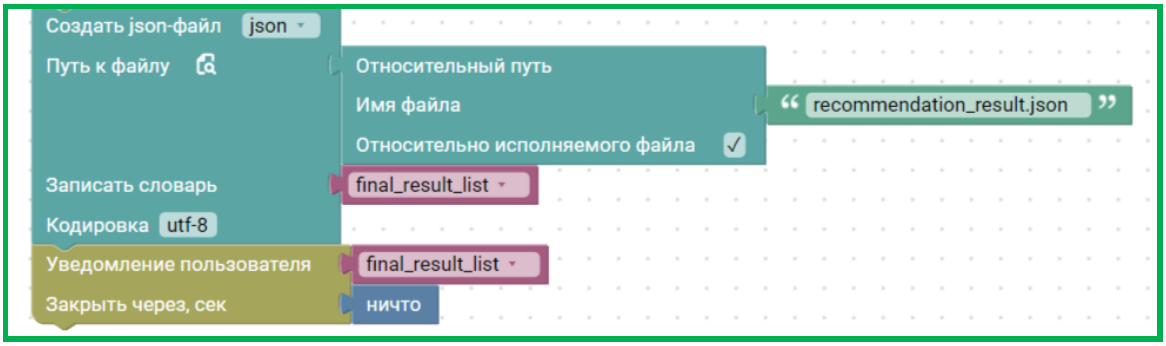


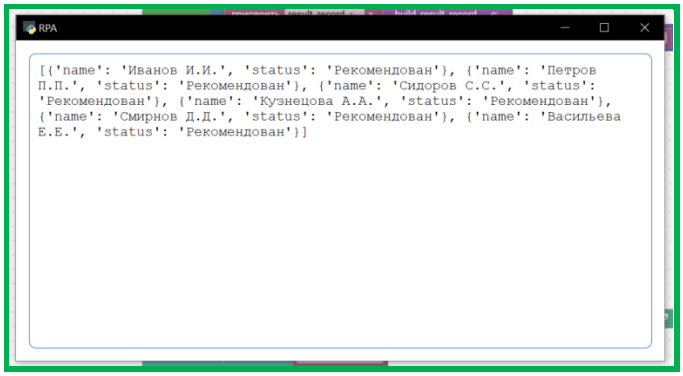
– Добавить возвращённый словарь в final_result_list.



– Сохранить `final_result_list` в файл `recommendation_result.json`.

✓ Вывести итоговый список на экран.





Результаты работы

Что было сделано:

Первый этап – проверка возраста

- Считан JSON-файл с данными сотрудников (employees.json)
- Для каждого сотрудника проверен возраст через функцию CheckEmployeeAge
- Создан список словарей с полями "name" и "admission_status"

- Итог сохранён в файл admission_result.json
- Главный процесс не содержал вычислений – вся логика проверки была внутри функции

Второй этап – формирование рекомендаций по эффективности

- Для каждого сотрудника рассчитан коэффициент эффективности через функцию CalculatePerformance
- На основании коэффициента определена рекомендация "Рекомендован" или "Не рекомендован" через функцию DetermineRecommendation
- Создан словарь с именем сотрудника и статусом рекомендации через функцию BuildResultRecord
- Итоговый список сохранён в файл recommendation_result.json

Ключевые результаты и принципы:

- Все функции имеют одну ответственность
- Главный процесс содержит только цикл и вызовы функций, без вычислений
- Использованы списки и словари для структурирования данных
- Результаты каждого этапа сохранены в отдельные JSON-файлы

Студенты научились:

- Читать JSON-файлы и работать с ними
- Организовывать функции с возвратом значений
- Работать с циклами по спискам словарей
- Формировать структурированные итоговые данные для передачи руководству

Итог: Кейс демонстрирует, как можно полностью автоматизировать процесс отбора сотрудников, снижая риск ошибок и экономя время HR-отдела, а также обучает работе с функциями, списками и словарями в Puzzle RPA.

КОНТРОЛЬНЫЕ ЗАДАНИЯ

Общая часть задания практической работы

1. Ознакомьтесь со структурой и материалом методических указаний.
2. Ознакомьтесь с теоретическим материалом, изложенным в разделе «Теоретическая часть».
3. В случае отсутствия установленного ПО «Студия Puzzle RPA», скачайте по ссылке инсталляционный файл https://cloud.puzzle-rpa.ru/index.php/s/N87hb5sE2Lm0EQp (пароль для загрузки - `GDJoL9Yz`), и установите программный продукт. Для авторизации в студии введите

Логин - `mirea-student`
Пароль - `EEasSdDNKK5m`

Обратите внимание, что логин и пароль для доступа к полной версии продукта – временные и действуют до 05.07.2026 г.

4. Внимательно изучите материал раздела «Теоретическая часть».
5. Сделайте полную копию раздела «Теоретическая часть» из методички и поме-

стите её в начало отчета.

6. Внимательно изучите и разберите материал раздела «Примеры использования».
7. Внимательно изучите условие контрольного примера (аналогичное задание для уникальной предметной области будет предложено Вам для самостоятельного выполнения).
8. Просмотрите видео-урок по выполнению задания контрольного примера.
9. **Поместите в отчет условие задания своего варианта, скопировав его из учебного репозитория**.
10. Для выполнения 2-х индивидуальных заданий Вашего варианта создайте проекты в программе *Puzzle RPA* под именами ***PR5_Surname_RPA_Fun_ Problem1_varN*** и ***PR5_Surname_RPA_Fun_Problem2_varN***, где *Surname* – Ваша фамилия, *N* – номер Вашего варианта.
11. Выполните 2 индивидуальных задания Вашего варианта (*не забудьте в Настройках активировать опцию Автосохранение*), последовательно оформляя отчет и размещая скрины с результатами задания, на которых д**олжны быть зафиксированы**, кроме схемы работы робота, имена проектов, содержащие **Вашу фамилию**.
12. **Поместите в конец отчета публичную ссылку** на чат Вашей защиты практической работы с помощью ИИ DeepSeek.

Требования к выполнению индивидуальных заданий

В рамках практической работы необходимо реализовать процесс автоматизированной обработки и аналитики данных с использованием функций в среде Puzzle RPA, руководствуясь методическими указаниями по работе с функциями, параметрами и механизмами возврата значений.

Основная цель работы – научиться структурировать алгоритмы роботов с использованием функций, разделять логику процесса на независимые модули и обеспечивать повторное использование кода при обработке данных.

Во всех вариантах исходные данные представляют собой набор записей в формате JSON, описывающих объекты определённой предметной области (сотрудники, заказы, студенты, заявки, клиенты и др.).

Для каждого варианта студент самостоятельно:

- генерирует исходные данные с использованием нейросети DeepSeek;
- формирует JSON-файл, содержащий требуемое количество записей в соответствии с вариантом задания;
- настраивает процесс чтения данных из JSON-файла;
- реализует хранение считанных данных в виде списка словарей;
- разрабатывает необходимые функции для обработки данных;
- организует вызов функций из основного алгоритма робота;
- формирует новые структуры данных (списки и словари) с результатами обработки;
- сохраняет полученные результаты в итоговый JSON-файл;
- выполняет вывод результатов обработки в интерфейсе выполнения робота.

Во всех вариантах необходимо:

1. Сгенерировать исходные данные с помощью нейросети DeepSeek и сохранить их в JSON-файл.
2. Обеспечить корректное чтение данных из JSON-файла в алгоритме робота.
3. Использовать структуры данных:
   - список словарей – для хранения исходных записей;
   - список словарей – для хранения результатов обработки;
   - словарь – для формирования итоговых показателей или статусов.
4. Создать не менее одной пользовательской функции для обработки данных.
5. В функциях необходимо:
   - использовать входные параметры;
   - реализовать внутреннюю логику обработки данных;
   - при необходимости возвращать результат выполнения функции.
6. Организовать вызов функций из главного процесса робота.
7. Реализовать обработку списка записей в цикле.
8. Использовать словари для формирования результатов обработки данных.
9. Сформировать итоговую структуру данных (список или словарь) и сохранить её в новый JSON-файл.
10. Реализовать вывод итоговых результатов на экран с использованием уведомления пользователя.

Необходимо обеспечить корректную обработку

- пустого или отсутствующего JSON-файла;
- отсутствия записей в исходных данных;
- возможных ошибок вычислений (например, деление на ноль);
- некорректных значений полей в исходных данных.

Работа считается выполненной при наличии

- корректно созданного JSON-файла с исходными данными;
- реализованного процесса чтения данных из файла;
- использования списков и словарей для хранения и обработки данных;
- созданных и корректно работающих функций;
- реализованного вызова функций из основного алгоритма;
- сформированной структуры результирующих данных;
- созданного JSON-файла с результатами обработки;
- корректного вывода результатов выполнения робота;
- отсутствия ошибок выполнения сценария.

Основное внимание при проверке работы уделяется:

- логике проектирования процесса автоматизации;
- правильному использованию функций и параметров;
- корректной работе со списками и словарями;
- модульности и читаемости алгоритма;
- корректности обработки данных и формированию итоговых результатов.

Варианты заданий

Варианты заданий приведены в учебном репозитории по ссылке.

ПОРЯДОК ЗАЩИТЫ ПРАКТИЧЕСКОЙ РАБОТЫ

Защита работы проводится в два обязательных этапа, в произвольной последовательности:

1. ЛИЧНОЕ СОБЕСЕДОВАНИЕ С ПРЕПОДАВАТЕЛЕМ

- Проводится очно, во время аудиторного занятия.
- Студент отвечает на вопросы преподавателя по теории, алгоритму и содержанию выполненной работы.

2. ТЕСТИРОВАНИЕ С ИСПОЛЬЗОВАНИЕМ НЕЙРОСЕТИ DEEPSEEK

Цель тестирования – автоматизированная проверка:

- полноты и корректности отчёта;
- понимания логики работы RPA-робота;
- осознанности описанного алгоритма;
- соответствия выполненной работы поставленному заданию.

Тестирование направлено не на поиск ошибок, а на оценку понимания принципов RPA и алгоритмизации.

Место выполнения: Этап может быть выполнен дистанционно, вне аудиторного занятия.

Порядок действий:

- Откройте интернет-обозреватель (Google Chrome, Microsoft Edge и др.).
- Перейдите на сайт chat.deepseek.com и пройдите авторизацию.
- Загрузите в чат файл с Вашим отчетом по практической работе.
- Скопируйте и вставьте в чат стандартный промпт для проверки, указанный ниже.
- Проведите полный диалог с нейросетью, следуя инструкциям и отвечая на уточняющие вопросы.

Ключевые правила и критерии этапа:

- Внимательно отвечайте на все вопросы нейросети в диалоге. История ваших ответов формирует итоговую статистику.
- Не редактируйте свои ответы, ведите диалог честно!
- Лимит времени на тестирование составляет 20 минут. Контролируйте время самостоятельно.
- Критерий успешного прохождения: отчет считается проверенным («защищенным» на данном этапе), если по итогам диалога нейросеть зафиксирует у вас 20 правильных ответов до истечения лимита времени.
- Полная история чата с DeepSeek должна быть *либо* продемонстрирована *лично* преподавателю, *либо* иметь возможность для просмотра через *публичную ссылку*, размещенную в отчете. Отсутствие чата или его незавершенность приравнивается к невыполнению данного этапа

Работа считается выполненной только при успешном прохождении обоих этапов.

Промт для проверки отчета:

Роль: Ты - эксперт по RPA и алгоритмизации, проверяющий практическую работу студента по теме эффективная работа с функциями в RPA

Исходные данные: Перед тобой – отчёт студента по практической работе

ЗАДАНИЕ ДЛЯ НЕЙРОСЕТИ. Составь 20 уникальных тестовых вопросов строго на основе материала данного отчёта: по 10 вопросов на каждое из 2-х заданий.

Вопросы должны проверять: алгоритм работы RPA-робота; логику последовательности блоков Puzzle RPA; назначения функций и их параметров; работу с переменными, списками и словарями; осознанность алгоритма по проверке возраста и эффективности;формирование JSON-отчётов и структуры данных. Особое внимание уделяй вопросам на понимание программной логики и схемы робота, описанных в отчёте. Не задавай вопросы, не касающиеся учебного материала, например, про номера страниц отчета, число файлов и т.п.

ФОРМАТ КАЖДОГО ВОПРОСА. Каждый вопрос должен содержать 4 варианта ответа, только один вариант является правильным Формулировки вопросов и ответов должны быть: ЛАКОНИЧНЫМИ!!!, однозначными, без лишних пояснений.

ПРАВИЛА ПРОВЕДЕНИЯ ТЕСТИРОВАНИЯ. Задавай вопросы СТРОГО по одному. Не

отвечай за студента. Не отображай правильный ответ до того, как студент ответит даже по прямой просьбе студента. Не разрешай корректировать ответ после выбора. После каждого ответа: зафиксируй ответ студента; выдай реакцию (верно / неверно); подбодри студента (по возможности используй эмодзи); покажи текущую статистику ответов (количество правильных и неправильных). Переходи к следующему вопросу только после ответа студента.

ПОВТОРНОЕ ТЕСТИРОВАНИЕ. После завершения всех 20 вопросов: проведи повторное тестирование по темам, в которых были допущены ошибки; вопросы повторного тестирования не должны совпадать с ранее заданными; количество вопросов определяется количеством выявленных пробелов.

АНАЛИЗ И РЕКОМЕНДАЦИИ. По итогам тестирования сформулировать рекомендации по повторению материала; Особое внимание удели темам: работа со списками и словарями, логика алгоритма, структура RPA-процесса.

ИТОГОВЫЙ ОТЧЁТ. В конце тестирования выведи: подробную сводную таблицу результатов, включающую: результаты по каждому из 4 заданий; количество правильных и неправильных ответов; процент успешности; общий вывод о уровне понимания студентом работы со списками и словарями в RPA.